Indian Institute of Technology Bombay
Department of Computer Science and Engineering

Structured LLM Optimization

Systems for LLMs



Saksham Rathi

Under the guidance of

Prof. Mythili Vutukuru

Bachelor's Thesis Project

Academic Year: 2025–26

Contents

List of Figures 3

List of Tables 4

1 Background 5
- 1.1 Stages in the Model Creation Pipeline . . . 5
- 1.2 Transformer Architecture . . . 7
  - 1.2.1 Tokenization . . . 7
  - 1.2.2 Embedding and Attention . . . 7
  - 1.2.3 Multi-Head Masked Attention . . . 7
- 1.3 Inferencing . . . 7
- 1.4 LLM Inferencing . . . 7
  - 1.4.1 Forward Pass . . . 8
  - 1.4.2 Token Probability Distribution . . . 8
  - 1.4.3 Generation Loop . . . 8
  - 1.4.4 Modes of Inference . . . 9
- 1.5 Inference Engines . . . 9
  - 1.5.1 Optimizations in Inference Engines . . . 10
- 1.6 Prefill vs Decode . . . 10
- 1.7 Metrics . . . 11
  - 1.7.1 Throughput . . . 11
  - 1.7.2 Latency . . . 11
- 1.8 KV Caching . . . 11

2 Flash Attention 12

3 vLLM 13

4 SwiftLLM 14

5 SGLang 15

6 Initial Experiments 16
- 6.1 System Prompt in Cache Experiment . . . 16
  - 6.1.1 Example 1 . . . 16
  - 6.1.2 Example 2 . . . 17
  - 6.1.3 Observations . . . 18
  - 6.1.4 Analysis over different steps and using different metrics 19
  - 6.1.5 Profiling across pipeline stages . . . 19

6.2 Batch of Requests 20
6.3 Throughput and Latency Measurements across different techniques 20
6.4 LLM Request Timelines 23

7 QLora **25**

Bibliography **26**

List of Figures

1.1 Adopted from llm 5
1.2 Adopted from USE 6
1.3 Architecture of a Large Language Model (3) 8
1.4 Architecture of a Large Language Model (3) 9
1.5 Prefill vs Decode 10

6.1 Throughput 22
6.2 Average Latency 22
6.3 P95 Latency 22
6.4 20 Requests per second 23
6.5 25 Requests per second 23
6.6 All requests at once 23
6.7 Request Timeline with GPU Metrics 24

List of Tables

6.1 Profiler Output 19
6.2 Profiling results for various pipeline stages 19
6.3 Measuring the time taken to process a batch of requests using different techniques 20

Chapter 1

Background

Language models (LMs) are statistical models that predict the next word in a sequence given the previous words. They basically give us a probability distribution over words based on the context provided by the preceeding words. They enable machines to understand and generate human language. They can be used for a lot of tasks like answering questions, text summarization, translation, code generation and more.

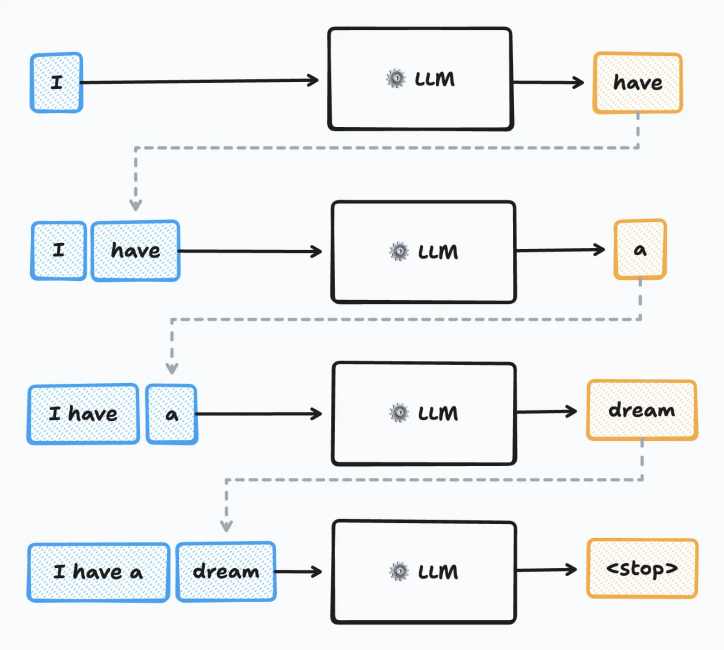


Figure 1.1: Adopted from llm

1.1 Stages in the Model Creation Pipeline

- **Data Preparation:** The initial stage involves gathering and preprocessing the training data, which can be categorized into two parts: (1) pretraining data, consisting of extensive unlabeled corpora obtained from public or private sources and curated through processes like detoxification and deduplication; (2) alignment data, comprising a smaller set of high-quality labeled corpora used to align the model with specific

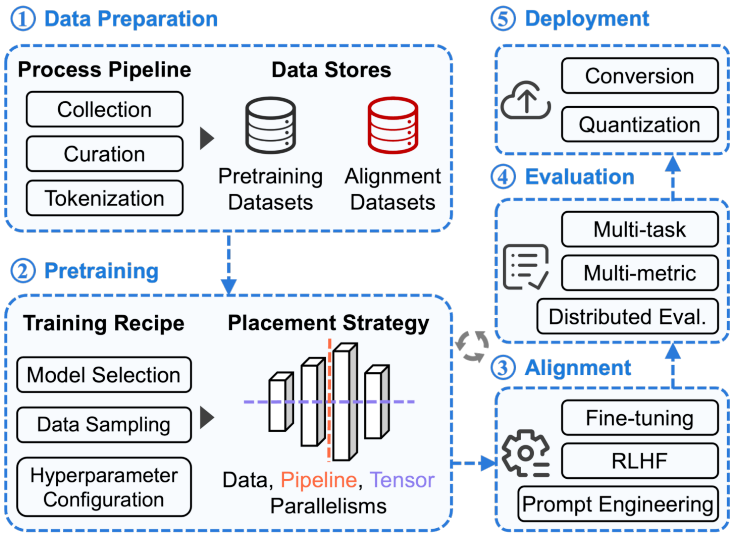


Figure 1.2: Adopted from USE

tasks. This data is typically acquired through expensive human annotation or labeling. Besides, all the data must be tokenized to ensure compatibility with the model's input.

- **Pretraining:** It involves self-supervised training on large-scale curated data, demanding a majority of resources within the overall development workflow. Training LLMs efficiently at scale necessitates various system innovations, such as state-sharding optimizers, meticulous model placement using data, pipeline, and tensor parallelisms.

- **Alignment:** This stage aims to adapt LLMs with user intent on a wide range of downstream tasks. Two primary aligning paradigms are commonly used: (1) prompt engineering, specifying prompts (i.e., inputs) without modifying model parameters. For example, in text summarization, appending a prompt "TL; DR" to the input article can improve model performance; (2) fine-tuning, updating model parameters on a task-specific dataset to improve performance in a particular domain.

- **Evaluation:** Given the vast application scenarios of LLM, it may be inaccurate to assess model quality solely based on a single metric like training loss. There are numerous factors to consider, such as accuracy, fairness, and toxicity. Consequently, it is crucial to account for a diverse set of criteria and measure performance across multiple tasks. Furthermore, regular evaluation is essential during the pretraining stage to provide timely feedback on model quality.

- **Deployment:** To meet the strict cost and latency constraints of LLM applications, several advanced techniques have been developed to achieve efficient model serving, including quantization, distillation, CUDA kernel optimization, model parallelism and memory management.

1.2 Transformer Architecture

1.2.1 Tokenization

Tokenization is the process of converting raw text into a sequence of tokens that the model can process. Instead of working directly with characters or words, modern language models often rely on subword tokenization methods such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece. These approaches break words into smaller, more frequent units, balancing vocabulary size and representation power. For example, the word "unhappiness" may be split into "un", "happiness", while rare words are decomposed into character-level units. Tokenization ensures consistent representation of unseen or rare words and significantly reduces out-of-vocabulary issues.

1.2.2 Embedding and Attention

Once tokenized, tokens are mapped into continuous vector representations called embeddings. These embeddings capture semantic and syntactic relationships among words, such that similar words are close in the embedding space. Positional encodings are added to embeddings to preserve word order information, since attention mechanisms are permutation invariant by default.

Attention is the core mechanism of transformer models. It allows the model to selectively focus on different parts of the input sequence when generating or predicting tokens. Given a query, attention computes weighted similarities with key-value pairs, enabling contextualized representations that consider both local and long-range dependencies.

1.2.3 Multi-Head Masked Attention

Masked attention is used in autoregressive language models to prevent the model from "looking ahead" at future tokens during training. This masking enforces causality by ensuring that each token can only attend to itself and preceding tokens. Multi-head attention extends this concept by computing multiple attention functions in parallel. Each head captures different aspects of relationships—such as syntactic dependencies, semantic similarity, or positional cues. The outputs of all heads are concatenated and linearly transformed, providing a richer, more expressive representation of the input.

1.3 Inferencing

1.4 LLM Inferencing

Large Language Model (LLM) inferencing refers to the process of using a trained model to generate predictions (such as text completions, answers, translations, etc.) given some input. Unlike training, where parameters are updated, inferencing only runs the trained model forward to produce outputs.

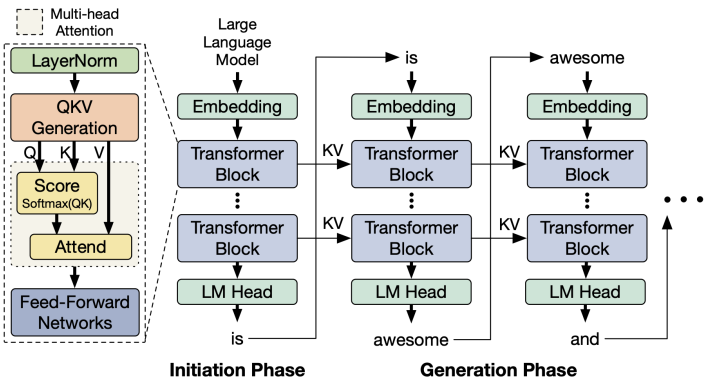


Figure 1.3: Architecture of a Large Language Model (3)

1.4.1 Forward Pass

- The forward pass is the core computation step in inferencing.
- Input tokens (converted to embeddings) are passed layer by layer through the transformer blocks:
  - Embedding → Attention → Feedforward → Normalization (repeated across layers).
- Each forward pass produces *logits*: a vector of unnormalized scores for the next token in the vocabulary.

1.4.2 Token Probability Distribution

- The logits are passed through a softmax to get probabilities.
- These probabilities define the model's belief about which token should come next.
- Different decoding strategies may be applied:
  - **Greedy decoding**: pick the token with maximum probability.
  - **Sampling**: sample a token according to the probability distribution.
  - **Top-k / Top-p sampling**: restrict sampling to the most probable tokens.
  - **Beam search**: keep multiple hypotheses alive and expand them.

1.4.3 Generation Loop

- Inferencing is auto-regressive:
  1. Generate one token.
  2. Append it to the context.
  3. Feed back into the model.

- Repeat until a stopping criterion (e.g., end-of-sequence token or maximum length) is reached.

1.4.4 Modes of Inference

- **Forward only** (logits for a given input): used in classification, scoring, and embeddings.
- **Generate** (auto-regressive token loop): used in chatbots, summarization, and translation.
- **Hybrid tasks**: e.g., retrieval-augmented generation (RAG), where inference integrates external knowledge.

1.5 Inference Engines

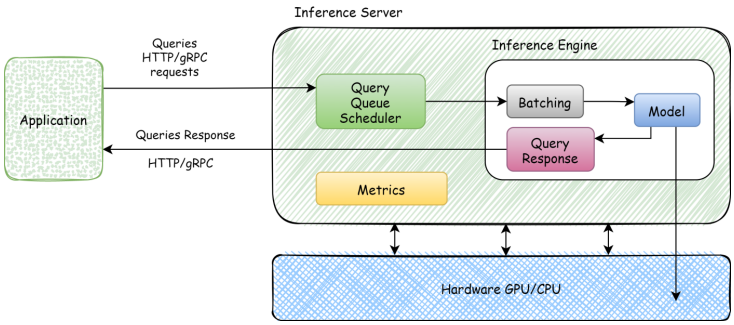


Figure 1.4: Architecture of a Large Language Model (3)

An **inference engine** is the software and systems layer that manages the execution of large language model (LLM) inference on hardware resources. While the model architecture and parameters define the computation, inference engines ensure that requests are executed efficiently, reliably, and at scale.

- **Batching:** Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead. Efficient batching is crucial for achieving high throughput in production environments.
- **Streaming:** Outputs are generated token-by-token and streamed back to the user as they are produced, reducing latency for interactive applications such as chatbots.
- **Fault Management:** Inference engines must be resilient to hardware or software failures. This includes checkpointing, task retries, and robust scheduling.
- **Error Handling:** Engines provide mechanisms for handling issues such as invalid inputs, out-of-memory errors, or timeouts without disrupting other active requests.

- **Parallelism (Multi-GPU):** Large models often cannot fit on a single GPU. Inference engines handle model parallelism and pipeline parallelism to distribute computation across multiple GPUs or nodes, while minimizing communication overhead.

1.5.1 Optimizations in Inference Engines

- **Memory Storage Optimizations:** Inference engines carefully manage GPU memory by caching key-value pairs (KV cache), reusing memory buffers, and applying quantization or weight-sharing techniques to reduce footprint.
- **Offloading:** Portions of the model state or intermediate activations can be offloaded to CPU or other storage when not actively needed on the GPU. This allows inference of models larger than GPU memory capacity.

1.6 Prefill vs Decode

LLM inference consists of two phases:

- **Prefill:** The input/prompt is ingested in parallel. All tokens are processed together to build the key-value (KV) cache, enabling the generation of the first output token.
- **Decode:** Subsequent tokens are generated auto-regressively. Each new token is computed sequentially using the KV cache from earlier steps.

In short, *prefill is parallel and one-time*, while *decode is sequential and repeated.*

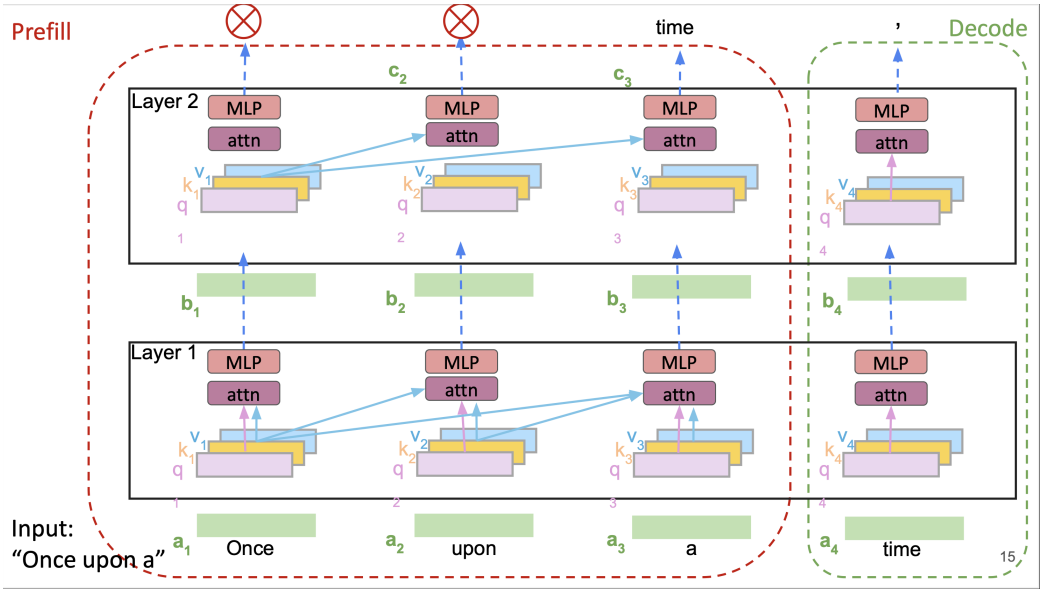


Figure 1.5: Prefill vs Decode

1.7 Metrics

The performance of LLM inference is typically measured using two key metrics:

1.7.1 Throughput

- Defined as the number of tokens processed per second.
- Reflects system efficiency and GPU utilization across multiple requests.

1.7.2 Latency

- Measures the responsiveness of the system.
- **Time to first token (TTFT):** Corresponds to the prefill phase, i.e., the delay before the first output token is produced.
- **Time between tokens:** Corresponds to the decode phase, i.e., the per-token delay during sequential generation.

1.8 KV Caching

- **Query (Q):** A representation of the current token, used to score its relevance against all past tokens. During decoding, we only compute the query for the token currently being processed.
- **Key (K):** Keys act like labels for tokens in the sequence. They are used to match queries and determine which past tokens are relevant.
- **Value (V):** Values are the actual token representations. Once relevance scores are computed (using Q and K), the corresponding values are aggregated to produce the output representation.

In causal self-attention, the attention for the current token depends on the keys (K) and values (V) of all past tokens.

- **Without caching:** K and V for all previous tokens are recomputed at every decoding step, causing redundant work.
- **With caching:** K and V are generated once during prefill/decoding and stored. At each new step, only the K and V for the latest token are computed, while past values are reused.

Chapter 2

Flash Attention

Chapter 3

vLLM

Chapter 4

SwiftLLM

Chapter 5

SGLang

Chapter 6

Initial Experiments

6.1 System Prompt in Cache Experiment

The basic application which we want to cater is inferencing in cases where the system prompt is common across multiple queries. And therefore, we want to utilize caching by already filling the KV-Cache with the system prompt, and then putting user queries on the top of it.

6.1.1 Example 1

I picked the 200-word summary of the demand paging lecture as the system prompt to be provided to the LLM. The model which we used was `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1.1 billion parameters). The system prompt provided was:

> The lecture discusses demand paging, a key concept in virtual memory management. Instead of loading all pages of a process into memory, only the required pages are brought in, while the rest may stay in swap space on disk. This allows processes to use address spaces larger than physical memory and enables efficient memory utilization.
> Each process has a page table, with entries containing a valid bit (indicating whether the page is part of the process) and a present bit (whether it is currently in memory). If valid=0, the process makes an illegal access. If valid=1 but present=0, a page fault occurs. The OS then allocates a memory frame, possibly evicts a victim page using a page replacement policy, updates the page table, and resumes execution.
> Victim pages may be file-backed or anonymous, and may be dirty (modified) or clean. Dirty pages require writing back to disk before eviction. Policies like FIFO and LRU determine which pages to evict; LRU is effective due to locality, while FIFO can suffer from Belady's anomaly.
> Excessive page faults can cause thrashing, where the system spends more time swapping pages than doing useful work. To prevent this, the OS ensures that each process's working set fits in memory.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.339 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

Query 1: What is the difference between a valid bit and a present bit?

Response 1: The difference between a valid bit and a present bit is that a valid bit indicates that a page is part of a process, while a present bit indicates that a page is currently in memory.

Query 2: Why does thrashing occur in demand paging systems?

Response 2: Thrashing occurs in demand paging systems because excessive page faults can cause the system to spend more time swapping pages than doing useful work. This is because demand paging systems allocate memory frames to processes based on their page table entries, which are either valid or present. If a process has a page fault, it must be allocated a memory

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.1 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.94 seconds.

6.1.2 Example 2

Let us take another example, this time with a summary of the semaphores lecture. The system prompt provided was:

The lecture introduces semaphores, a synchronization primitive similar to condition variables, used to coordinate threads. A semaphore maintains a counter, initialized to some value. The down (wait) operation decrements the counter and blocks the thread if it becomes negative, while the up (post) operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations.
Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They are also useful for synchronization between threads, e.g., enforcing execution order (T1 before T2) or ensuring two threads finish one step before starting another. Incorrect usage can cause deadlocks, such as when both threads wait on each other.
The producer-consumer problem is revisited using semaphores. Two semaphores track empty and filled buffer slots, while a binary semaphore (mutex) ensures mutual exclusion on buffer access. Misuse, like holding locks during blocking down calls, can also lead to deadlocks.
Guidelines emphasize careful initialization, ensuring every down has a corresponding up, and using separate semaphores for signaling and locking. The lecture also covers batched processing with semaphores, showing how request threads and a batch processor can coordinate efficiently.
Semaphores thus provide flexible yet tricky synchronization, requiring careful design to avoid deadlocks.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.347 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

Query 1: How does the down operation on a semaphore synchronize threads during execution?

Response 1: A semaphore is a synchronization primitive that maintains a counter, initialized to some value. The down operation decrements the counter and blocks the thread if it becomes negative. The up operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations. Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They

Query 2: Why is an extra binary semaphore (mutex) needed in producer-consumer implementation?

Response 2: An extra binary semaphore (mutex) is needed in producer-consumer implementation because the producer and consumer threads can access the same buffer at the same time. This can lead to deadlocks if the buffer is not properly protected.

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.774 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.957 seconds.

6.1.3 Observations

- As we can see, there is roughly an improvement of 0.2-0.7 seconds for an experiment of 2 queries, which is also intutively clear, because we are just passing over the system prompt once.
- As we increase the size of the system prompt, and the number of tokens we generate, the difference becomes even wider.
- The time taken to feed through the system prompt, and cloning the KV cache values, can be ignored safely because we can do this before we get the actual user queries.
- The first query of the second example didn't get a superior response, however this is independent from our experiment settings, and depends more on the type of model we use, for example increasing the number of parameters (using a larger model) would definitely help.

6.1.4 Analysis over different steps and using different metrics

I now use a profiler to actually figure out at what all places, the most amount of time is being spent:

Table 6.1: Profiler Output

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
aten::matmul	2.67%	56.256ms	30.07%	632.976ms	40.575us	0.000us	0.00%	589.952ms	37.817us	15600
aten::linear	1.16%	24.342ms	30.78%	647.989ms	41.806us	0.000us	0.00%	589.802ms	38.052us	15500
aten::mm	14.46%	304.341ms	22.68%	477.495ms	30.806us	579.871ms	69.41%	589.802ms	38.052us	15500
cutlass kernel (relu bf16)	0.00%	0.000us	0.00%	0.000us	0.000us	234.992ms	28.13%	234.992ms	65.385us	3594
ampere_bf16_128x64_sliced1x2	0.00%	0.000us	0.00%	0.000us	0.000us	124.274ms	14.87%	124.274ms	66.457us	1870
ampere_bf16_128x128_stages_32x1	0.00%	0.000us	0.00%	0.000us	0.000us	94.240ms	11.28%	94.240ms	44.621us	2112
aten::mul	6.55%	137.891ms	10.29%	216.715ms	10.728us	61.543ms	7.37%	61.560ms	3.048us	20200
aten::copy_	3.97%	83.637ms	8.32%	175.218ms	9.679us	50.846ms	6.09%	50.857ms	2.809us	18103
ampere_bf16_128x64_stages_64x3	0.00%	0.000us	0.00%	0.000us	0.000us	44.572ms	5.34%	44.572ms	19.481us	2288
aten::scaled_dot_product_attention	0.84%	17.683ms	5.51%	116.049ms	52.749us	0.000us	0.00%	44.127ms	20.058us	2200
aten::_scaled_dot_product_flash_attention	0.63%	13.224ms	4.67%	98.366ms	44.712us	0.000us	0.00%	44.127ms	20.058us	2200
aten::_flash_attention_forward	1.00%	21.077ms	3.49%	73.410ms	33.368us	44.073ms	5.28%	44.127ms	20.058us	2200
flash_fwd_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	43.833ms	5.25%	43.833ms	20.125us	2178
elementwise_mul_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	37.673ms	4.51%	37.673ms	2.833us	13300
ampere_bf16_64x64_sliced1x4	0.00%	0.000us	0.00%	0.000us	0.000us	34.059ms	4.08%	34.059ms	28.669us	1188
aten::clone	0.87%	18.407ms	6.48%	136.324ms	15.491us	0.000us	0.00%	26.201ms	2.977us	8800
elementwise_copy_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	26.198ms	3.14%	26.198ms	2.977us	8800
aten::add	4.45%	93.743ms	7.17%	150.912ms	11.347us	24.648ms	2.95%	24.657ms	1.854us	13300
aten::to	0.93%	19.568ms	8.33%	175.368ms	12.345us	0.000us	0.00%	24.656ms	1.736us	14205
aten::_to_copy	1.29%	27.125ms	7.40%	155.800ms	16.747us	0.000us	0.00%	24.656ms	2.650us	9303

Now we go over each of the rows, and further analyze the values:

1. `aten::matmul` = Pytorch's high level matrix multiplication. Runs on CPU side, but calls matrix multiply functions (`aten::mm`) on CUDA.
2. `aten::linear` = The linear layer, has large CUDA compute
3. `cutlass kernel (relu bf16)` = Stands for general matrix multiplication, entirely on CUDA
4. `ampere_...` = Specialized Ampere architecture GEMM kernel in BF16 precision for different tile sizes
5. `aten::mul` = Element-wise matrix multiplication, very high call count
6. `aten::copy_` = Copying tensors, very high call count
7. `aten::_flash_attention_forward` = The forward pass of flash attention

6.1.5 Profiling across pipeline stages

Stage	Time (s)
Tokenize system prompt	0.0052
System forward (KV cache fill)	0.3342
Tokenize query	0.0003
Per token (mean / first)	0.0148 / 0.0390
Generation total	0.7517
Decode tokens	0.0002

Table 6.2: Profiling results for various pipeline stages

6.2 Batch of Requests

We will measure the time taken to process a batch of requests, by varying the batch size:

Total number of queries: 24

Technique Used	**Time (s)**
Transformers (batch size = 1)	30.21
Transformers (batch size = 2)	19.51
Transformers (batch size = 3)	18.64
vLLM (no system prompt pre-filling)	0.45
vLLM (with system prompt pre-filling)	0.37
SGLang	1.03
NEO	0.55

Table 6.3: Measuring the time taken to process a batch of requests using different techniques

6.3 Throughput and Latency Measurements across different techniques

Here is the experiment setup:

- Model: `TinyLlama/TinyLlama-1.1B-Chat-v1.0`
- System Prompt: Roughly 1500 tokens (adopted from Ostep MLFQ schedulung chapter)
- Queries: 100 queries, each of length 10-20 tokens
- Max Output Length: 80 tokens
- Techniques Compared: vLLM (with and without automatic prompt caching), SGLang, swiftLLM

Here is how we generate the load:

Algorithm 1: Workload Generation at Target QPS

Input: LLM model `llm`, list of `prompts`, parameters `sampling_params`, target QPS `qps`
Output: Latencies of requests
Initialize empty list `latencies` ;
Set `interval` $\leftarrow 1/$`qps` ;
Set `start_time` $\leftarrow$ current time ;
Set index $i \leftarrow 0$;
Initialize empty `backlog` ;
Set `next_time` $\leftarrow$ current time ;
while $i <$ *length(`prompts`)* **do**
  Set `now` $\leftarrow$ current time ;
  Compute `intervals_passed` $\leftarrow \lfloor (now - next_time)/interval \rfloor + 1$;
  `// By the time we reached here, a few intervals might have passed, so we add that many prompts to the backload`
  for $k = 1$ **to** *`intervals_passed`* **do**
    if $i <$ *length(`prompts`)* **then**
      Append `prompts[i]` to `backlog` ;
      Increment i ;
  if *`backlog` is not empty* **then**
    Set `batch_start` $\leftarrow$ current time ;
    Call `llm.generate(backlog, sampling_params)` ;
    Set `batch_latency` $\leftarrow$ current time $-$ `batch_start` ;
    `// All the requests in the backloag have the same latency because they were processed in a batch`
    foreach *request in `backlog`* **do**
      Append `batch_latency` to `latencies` ;
    Clear `backlog` ;
  Update `next_time` $\leftarrow$ `next_time` + `intervals_passed` $\times$ `interval` ;
  `// If we are ahead of the schedule, we sleep until the next request time`
  Set `sleep_time` $\leftarrow \max(0, next_time -$ `current time`$)$;
  `sleep(sleep_time)` ;
return (`latencies`) ;

We primarily measure the following metrics:

- **Throughput (QPS)**: Total number of requests / Total time taken
- **Average Latency (s)**: Average time taken per request, basically the time a user will have to wait
- **P95 Latency (s)**: 95th percentile latency across all requests, this helps us understand the tail latency, which in turn blocks the next batch of requests

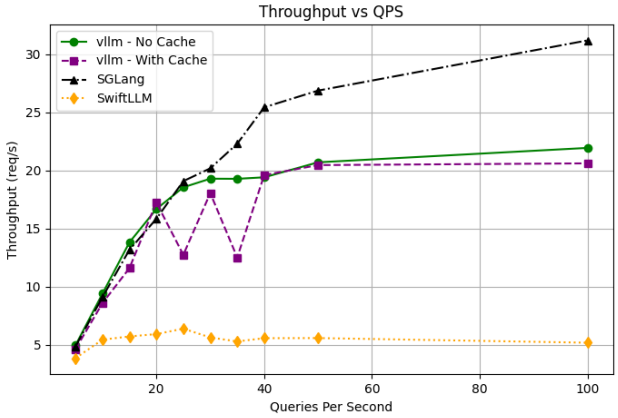


Figure 6.1: Throughput

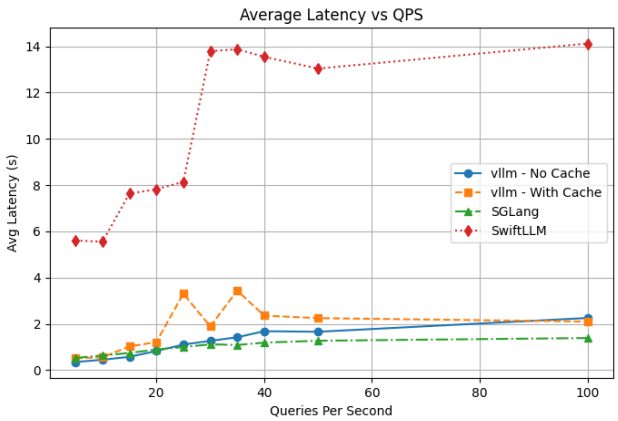


Figure 6.2: Average Latency

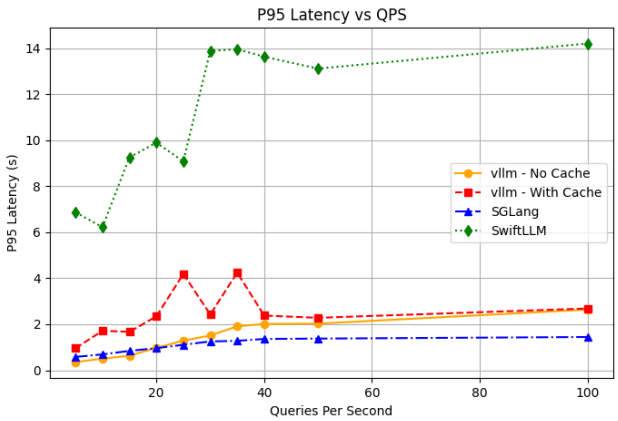


Figure 6.3: P95 Latency

6.4 LLM Request Timelines

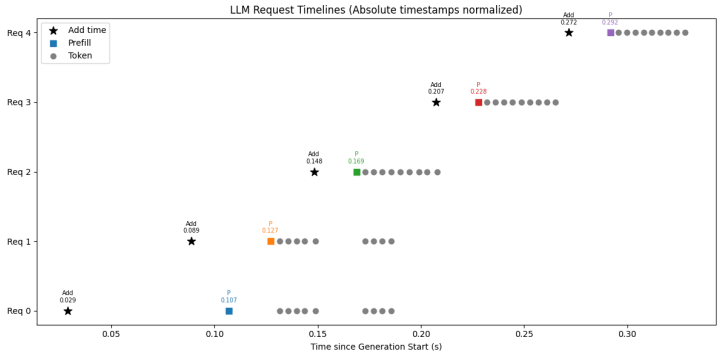


Figure 6.4: 20 Requests per second

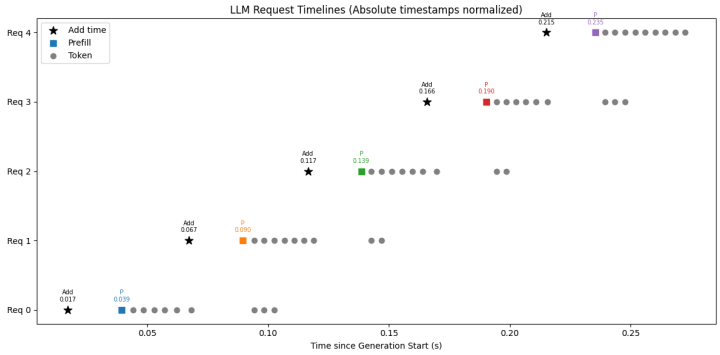


Figure 6.5: 25 Requests per second

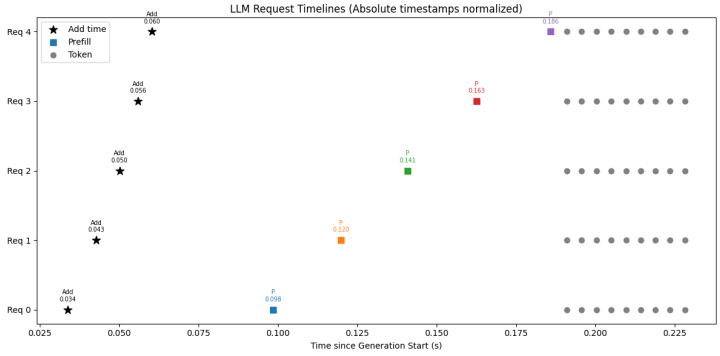


Figure 6.6: All requests at once

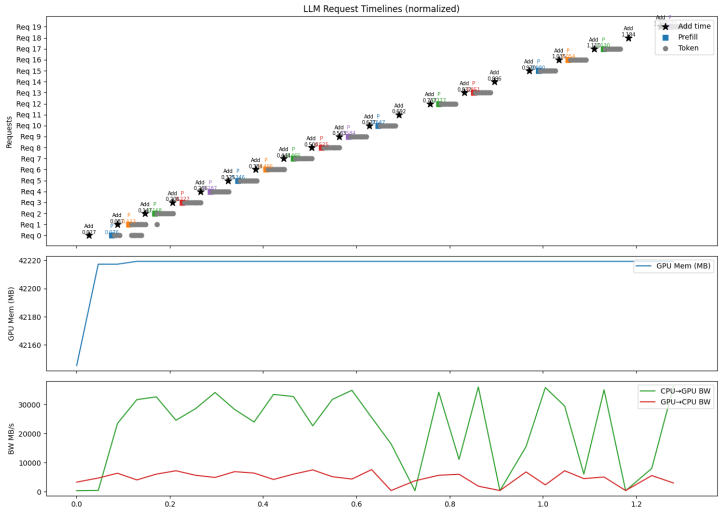


Figure 6.7: Request Timeline with GPU Metrics

Chapter 7

QLora

Bibliography

Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale. URL https://arxiv.org/pdf/2408.05499.

Understanding the workload characteristics of large language model development. URL https://www.usenix.org/publications/loginonline/understanding-workload-characteristics-large-language-model-development.

How llms generate text for the rest of us. URL https://pm.dartus.fr/posts/2025/how-llm-generate-text/.